CRC Error Detection

A Computer Networks Mini Project

Project Report

Submitted in partial fulfilment of the requirements for

Bachelor of Engineering in Computer Science and Engineering

Submitted by

[Student names and register numbers]

Under the guidance of

[Guide name and designation]

Department of Computer Science and Engineering

[College name]

[University name]

[Academic year]

Certificate and Declaration

Certificate template

This is to certify that the mini project entitled CRC Error Detection is submitted by [student names and register numbers] to the Department of Computer Science and Engineering, [college name], during [academic year]. The work is to be reviewed by the project guide and department before certification.

Project guide signature _____

Head of department signature _____

Internal examiner _____

External examiner _____

Place _____ Date _____

Student declaration template

We declare that the submitted version accurately describes the project implementation, the sources consulted and the testing performed. We will acknowledge assistance and external material according to our institution's academic requirements.

Student names and signatures _____

Submission note: Replace all bracketed fields, review the declaration against your actual work, and obtain genuine signatures. This page is an unsigned template, not an issued certificate.

Abstract and Contents

Abstract

This project implements an educational Cyclic Redundancy Check simulator in Java. It demonstrates how a sender generates redundant check bits, how a simulated communication channel can change selected bits, and how a receiver uses modulo 2 polynomial division to detect corruption. A Swing interface accepts binary data and a generator polynomial, displays the transmitted and received frames, and records XOR division steps.

The implementation uses MSB first augmented division with zero initial state, without reflection or final XOR. Automated tests passed 5128 checks, including an independent BigInteger polynomial oracle. Interface checks exercised the main workflow through offscreen rendering. The simulator demonstrates both detected and undetected errors; it does not transmit packets over a real network or correct damaged data.

Contents

1 Introduction and objectives
2 CRC theory and error detection properties
3 Worked example and system design
4 Requirements and implementation
5 Testing and validation
6 Interface outputs
7 Installation and demonstration
8 Conclusion and future enhancements
Appendix A Core source code
Appendix B Viva questions and references

1 Introduction and Objectives

Problem statement

During digital communication, a received bit sequence may differ from the sequence sent. Students often learn the CRC procedure as a calculation without seeing how a changed bit affects verification. This project connects the calculation to an interactive sender, channel and receiver simulation.

Aim

To develop a Java application that generates CRC check bits for binary data, simulates transmission errors and verifies the received frame using modulo 2 division.

Objectives

- Accept binary data and a valid generator polynomial.
- Generate a systematic codeword containing the original data followed by CRC bits.
- Simulate single and multiple bit changes at user selected positions.
- Display the receiver syndrome and an accurate detection result.
- Show XOR operations and allow the calculation log to be exported.
- Validate the algorithm with independent computations and boundary cases.

Project scope

The channel is represented by an editable received frame. No sockets, physical network, database or external service are needed. Positions start at 1 from the left. Leading zeros are preserved because frames are represented as strings. Data is limited to 256 bits for readable classroom traces, while the generator has 2 to 33 bits.

Learning outcomes

The project develops understanding of polynomial arithmetic over GF(2), redundant coding, input validation, modular Java design, Swing event handling and systematic testing. It also separates error detection from correction and authentication.

2 CRC Theory and Error Detection Properties

CRC treats a bit sequence as polynomial coefficients over GF(2). For example, 10011 represents x^4 + x + 1. The leading 1 is explicitly included in the generator entered into this application. A generator containing r + 1 bits has degree r and produces r CRC bits. [1]

Modulo 2 arithmetic

Addition and subtraction are both XOR. There are no carries or borrows: 0 XOR 0 = 0, 0 XOR 1 = 1, 1 XOR 0 = 1 and 1 XOR 1 = 0. Polynomial long division repeatedly aligns the generator with the current leading 1 and applies XOR.

Sender and receiver equations

Let D(x) be the data and G(x) the generator of degree r. The sender divides D(x)x^r by G(x) and obtains remainder R(x). The transmitted polynomial is T(x) = D(x)x^r + R(x), which is divisible by G(x). The receiver divides the received frame by G(x) without appending additional zeros. Its remainder is the syndrome.

A nonzero syndrome means an error was detected. A zero syndrome means no error was detected. If the error polynomial is a multiple of G(x), the damaged frame can still have a zero syndrome. Therefore a zero result cannot guarantee that the frame is unchanged. [1]

Detection properties

- For generators accepted here, all single bit errors are detected because the constant term is 1 and the polynomial has at least two nonzero terms.
- Every burst error of length at most r is detected. Burst length is measured from the first altered bit to the last, inclusive.
- All odd numbers of bit errors are detected only if G(x) contains the factor x + 1. This is not guaranteed for every accepted generator.
- Detection of other patterns depends on the generator and frame length. CRC is not a cryptographic integrity mechanism.

Implementation convention

This project uses a direct teaching model: MSB first, zero initial state, data augmented by r zero bits and no final XOR. Named industrial CRC variants may use different initial states, reflection and final processing, so matching a polynomial alone does not establish compatibility.

3 Worked Example and System Design

Item	Value
Data	1101011011
Generator	10011
Degree and CRC width	4
Data with four zeros	11010110110000
CRC remainder	1110
Transmitted frame	11010110111110

Sender calculation

```
Start       11010110110000
XOR at 1    01001110110000
XOR at 2    00000010110000
XOR at 7    00000000101000
XOR at 9    00000000001110
```

The final four bits are 1110. Appending them to the original data gives 11010110111110. Dividing this complete frame by 10011 produces 0000.

Error cases

Flipping bit 1 produces 01010110111110. The syndrome is 1101, so the receiver reports ERROR DETECTED. Flipping bits 1, 4 and 5 of the original frame produces 01001110111110. The error pattern is 10011000000000, which is 10011 shifted by nine places. It is a generator multiple, so the syndrome remains 0000 despite corruption.

System components

Component	Responsibility
Sender	Validate data and generator; append zeros; calculate CRC.
Simulated channel	Copy, edit or flip positions in the received frame.
Receiver	Divide received frame and interpret the syndrome.
Interface and log	Display frames, operations and results; export text.

4 Requirements and Implementation

Requirement	Project choice
Language and toolkit	Java 17 or newer; Java Swing
Operating environment	Windows, Linux or macOS with a graphical desktop
Dependencies	Full JDK; no external Java libraries or database
Practical machine	Any machine that runs the selected JDK and a desktop
Input limits	1 to 256 data bits; 2 to 33 generator bits

Sender algorithm

1. Read data and generator; reject invalid binary input.

2. Require the generator to start and end in 1.

3. Append generator length minus one zero bits to the data.

4. Scan the working array from left to right. When the current bit is 1, XOR the generator into the aligned segment.

5. Extract the last r bits and append them to the original data.

Receiver algorithm

1. Require a generated frame and the same generator used by the sender.

2. Check that received input is binary and has the transmitted frame length.

3. Divide the received frame directly using the same XOR routine.

4. Report ERROR DETECTED for a nonzero syndrome; otherwise report NO ERROR DETECTED.

Code organisation and complexity

CRC.java contains reusable calculation and bit flip methods. CRCApp.java contains the interface. CRCTest.java supplies algorithm tests and CRCUIEvidence.java checks the interface and creates offscreen captures. With frame length n and generator length m, basic division costs O(nm) time and O(n) working memory. If traces are retained, up to O(n) full frame snapshots require O(n squared) characters. The short input limit keeps demonstrations manageable.

Changing the generator after generation blocks verification until a new frame is generated. Invalid or duplicate flip positions are rejected. Export uses UTF 8 text and asks before overwriting an existing file.

5 Testing and Validation

Validation executed in the supplied environment on OpenJDK 17. The test suite completed with PASS: 5128 checks. The independent oracle uses BigInteger XOR and degree alignment instead of the production character array routine.

Test	Expected outcome	Observed
Known vector	CRC 1110 and expected frame	Pass
Unchanged frame	Syndrome 0000	Pass
Each of 14 single bit flips	Nonzero syndrome	14 passed
Flip bits 1 4 5	Changed frame with syndrome 0000	Pass
All zero data	Preserve all leading zeros	Pass
2550 data and generator pairs	Match independent CRC and verify	5100 checks passed
Ten invalid input cases	Reject invalid values	10 passed

Coverage details

Exhaustive data values of lengths 1 through 8 were tested for generators 11, 101, 1011, 10011 and 11001. Each of 2550 combinations produced two checks: CRC agreement with the oracle and a zero syndrome for the encoded frame. Remaining checks cover the known vector, all single bit locations, leading zeros, a deliberate undetected error and invalid inputs.

Interface verification

The actual Swing panel was exercised headlessly: generate and verify a clean frame, inject and detect a single bit error, demonstrate an undetected pattern, reject a changed generator and reset the form. Three offscreen renders are reproduced on the following pages. These are captures of the real component, not drawings of a proposed interface.

Validation limits

A native Windows desktop launch and interaction with the file chooser were not tested here. Before college demonstration, run run.bat on the presentation laptop, verify all three example cases and test Export log. Test logs are supplied in the evidence folder; no throughput or real network performance measurement is claimed.

6 Interface Outputs

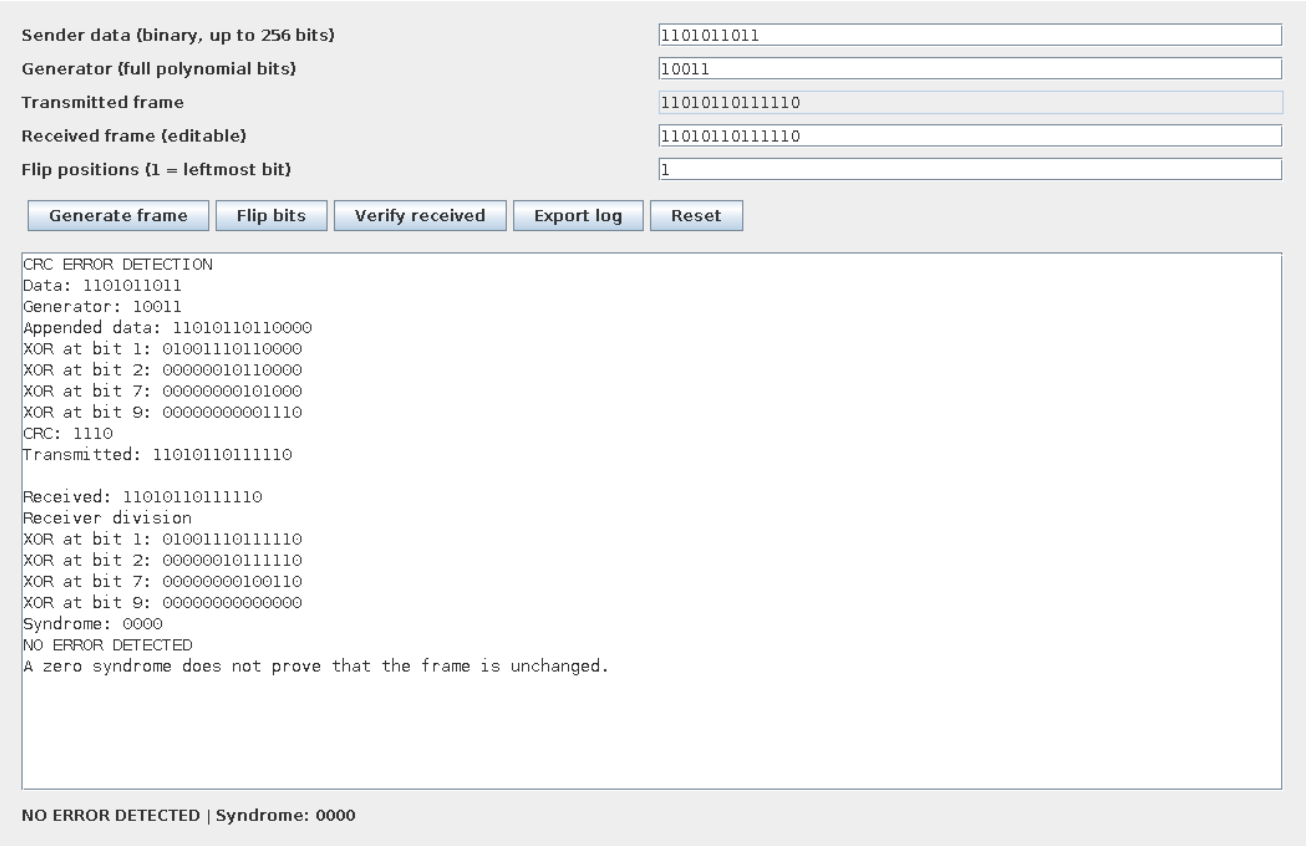


Figure 1 Clean frame verification using data 1101011011 and generator 10011. CRC 1110 is appended and the receiver syndrome is 0000.

Execution procedure

Enter data 1101011011 and generator 10011. Click Generate frame and then Verify received without changing any bit. The captured application output shows the sender division, CRC 1110 and transmitted frame 11010110111110.

Observed receiver result

The receiver divides the complete frame and obtains syndrome 0000. The status line displays NO ERROR DETECTED. The output is generated by executing the actual Swing panel; it is not a sample screen illustration. A zero syndrome means the CRC check passed, rather than proving that every possible error has been excluded.

6 Interface Outputs Detected Error

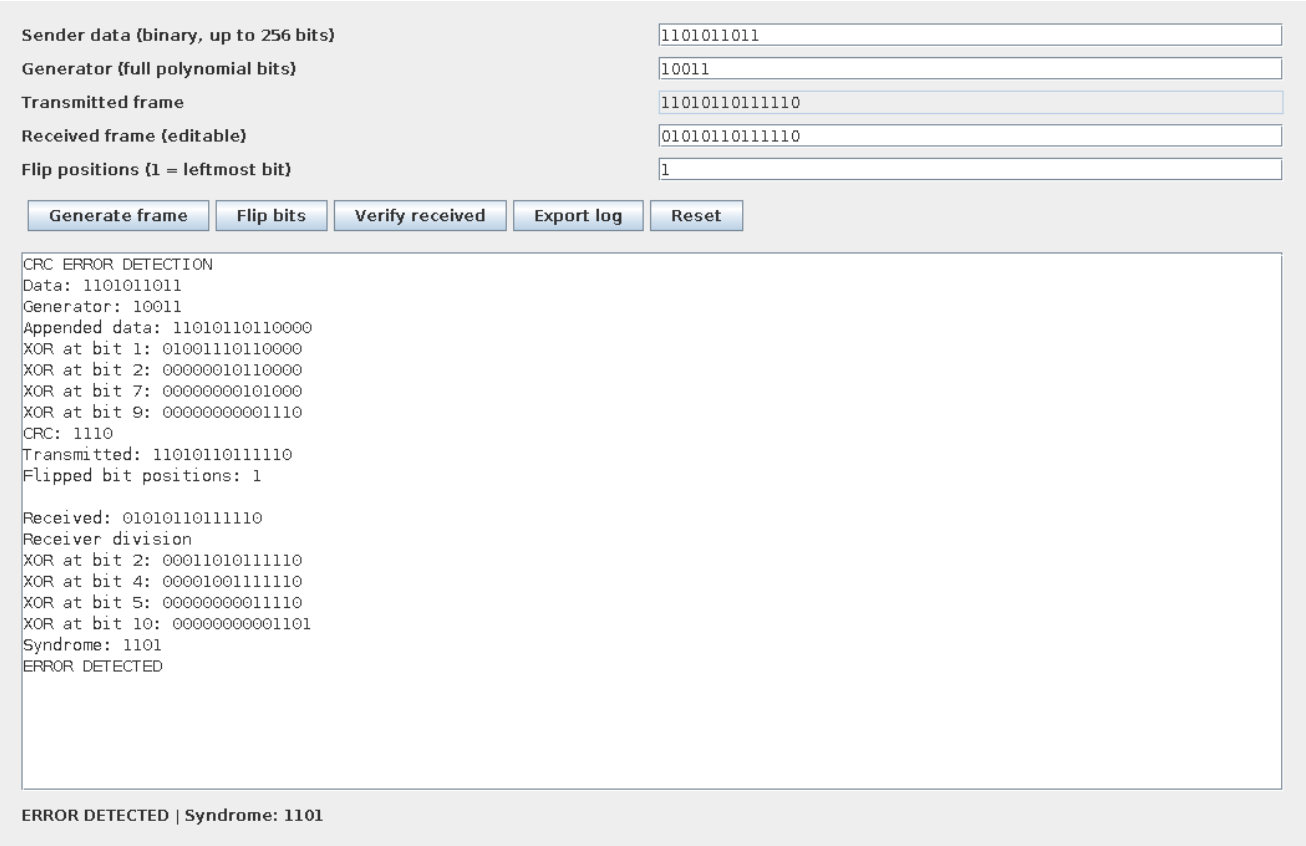


Figure 2 Bit 1 is flipped. The received frame differs from the transmitted frame, and the interface reports ERROR DETECTED with syndrome 1101.

Execution procedure

Generate the default frame again. Enter 1 in Flip positions, click Flip bits and then Verify received. Positions are counted from the left starting at 1. The first transmitted bit changes from 1 to 0, giving received frame 01010110111110.

Observed receiver result

The captured output shows a nonzero syndrome of 1101 and the status ERROR DETECTED. The receiver detects corruption through polynomial division. It does not repair the changed bit. In a communication protocol, a separate retransmission mechanism could request the message again.

6 Interface Outputs Continued

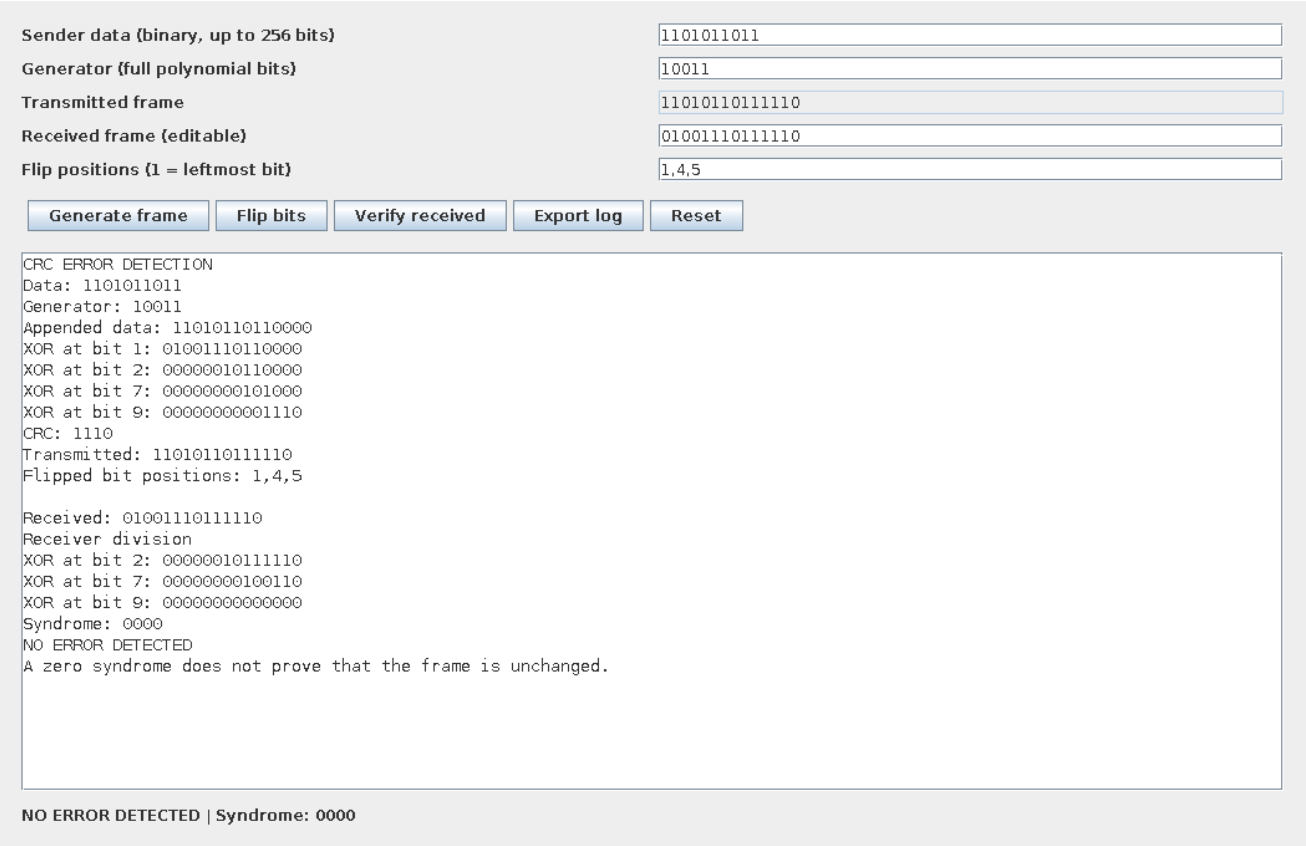


Figure 3 Bits 1, 4 and 5 are flipped from the original frame. The syndrome is 0000, showing that some error patterns are undetectable with this generator.

Recorded test output

```
PASS: 5128 checks
Data=1101011011 Generator=10011 CRC=1110 Frame=11010110111110
Clean syndrome=0000
Bit 1 flipped=01010110111110 Syndrome=1101
Bits 1,4,5 flipped=01001110111110 Syndrome=0000
```

Interpretation

The application does not compare the received frame to the saved original to decide its CRC result. Verification uses only polynomial division with the selected generator. This reproduces the limitation of CRC checking rather than concealing an undetected error by comparing against sender data.

7 Installation and Demonstration

Windows installation

1. Install a full JDK 17 or newer on the demonstration laptop.

2. Open Command Prompt and confirm java -version and javac -version.

3. Extract CRC_Error_Detection_Project.zip completely.

4. Open the extracted crc_project folder and double click run.bat.

5. If a compiler is unavailable, check that the JDK bin folder is on PATH.

Manual build and launch

```
javac -d build src/*.java
java -cp build CRCApp
```

Run these commands from the project folder. On Linux or macOS, sh run.sh performs the same steps. The graphical application needs a desktop session; automated calculation tests can run without one. Java compiler documentation explains how source is compiled into JVM class files. [2]

Suggested college demonstration

1. Explain the sender, channel and receiver roles.

2. Generate with the default values and identify CRC 1110.

3. Verify without changes and explain the zero syndrome.

4. Flip bit 1 and verify the nonzero syndrome.

5. Generate again, flip 1,4,5 and explain the undetected error.

6. Export the trace and show the XOR calculation log.

Reproduce the tests

```
java -cp build CRCTest
java -Djava.awt.headless=true -cp build CRCUIEvidence evidence
```

These commands assume the source has already been compiled. Capture your own native laptop output if your department requires full desktop screenshots.

8 Conclusion and Future Enhancements

The project provides a working Java simulator for CRC based error detection. It calculates check bits, creates a transmitted frame, simulates selected bit changes and verifies the received frame using the same polynomial. The XOR trace makes the calculation visible, while independent tests support the implementation's correctness within the stated scope.

The clean and corrupted frame examples demonstrate the intended behaviour. The generator multiple example shows why a zero remainder must be described as no error detected. The project therefore presents both the usefulness and the limitations of CRC.

Advantages

- Runs offline with standard Java libraries.
- Preserves binary strings and leading zeros.
- Supports custom generator polynomials and multiple bit changes.
- Includes a trace, reproducible tests and editable documentation.

Limitations

The simulator does not implement sockets, retransmission, error correction, byte file processing or a named standard CRC variant. It does not establish sender identity or protect against deliberate modification. Very long messages are outside the interface limits because the complete trace grows rapidly.

Future enhancements

- Add named CRC variants with fully specified initial state, reflection and final XOR settings.
- Extend the application to byte files and compare against known reference vectors.
- Add a socket based sender and receiver with a separate channel simulator.
- Demonstrate automatic repeat request after detected corruption.
- Compare simple parity, additive checksum and CRC on a defined set of error patterns.

Appendix A Core Source Code

The following listing is the reusable CRC engine. Complete interface, evidence and test source files are included in the project archive.

```
/** Educational systematic CRC: MSB first, zero initial state, no final XOR. */
public final class CRC {
    private CRC() {}
    public static void bits(String value, String label) {
        if (value == null || !value.matches("[01]+"))
            throw new IllegalArgumentException(label + " must contain only 0 and 1.");
    }
    public static void generator(String g) {
        bits(g, "Generator");
        if (g.length() < 2 || g.length() > 33 || g.charAt(0) != '1'
                || g.charAt(g.length()-1) != '1')
            throw new IllegalArgumentException("Generator: 2 to 33 bits, starting and ending with
1.");
    }
    public static String remainder(String input, String g, StringBuilder trace) {
        bits(input, "Input"); generator(g);
        if (input.length() < g.length()-1)
            throw new IllegalArgumentException("Frame is shorter than the CRC width.");
        char[] work = input.toCharArray();
        for (int i=0; i<=work.length-g.length(); i++) {
            if (work[i]=='1') {
                for (int j=0; j<g.length(); j++)
                    work[i+j] = work[i+j] == g.charAt(j) ? '0' : '1';
                if (trace != null) trace.append("XOR at bit ").append(i+1)
                    .append(": ").append(work).append('\n');
            }
        }
        return new String(work, work.length-g.length()+1, g.length()-1);
    }
    public static String encode(String data, String g, StringBuilder trace) {
        bits(data, "Data"); generator(g);
        return data + remainder(data + "0".repeat(g.length()-1), g, trace);
    }
    public static boolean passes(String frame, String g) {
        return !remainder(frame, g, null).contains("1");
    }
    /** One-based bit positions from the left; duplicates are rejected. */
    public static String flip(String frame, String positions) {
        bits(frame, "Frame");
        if (positions.trim().isEmpty()) throw new IllegalArgumentException("Enter bit positions, e.g.
1,4.");
        char[] out = frame.toCharArray();
        java.util.Set<Integer> seen = new java.util.HashSet<>();
        for (String token : positions.split(",", -1)) {
            int p;
            try { p=Integer.parseInt(token.trim()); }
            catch (NumberFormatException e) { throw new IllegalArgumentException("Use comma-separated
whole numbers."); }
            if (p<1 || p>out.length || !seen.add(p))
                throw new IllegalArgumentException("Positions must be unique and between 1 and " +
out.length + ".");
            out[p-1] = out[p-1]=='0' ? '1' : '0';
        }
        return new String(out);
    }
}
```

Appendix B Viva Questions and References

What is CRC? A redundant error detection code based on modulo 2 polynomial division.

Why are zeros appended? To multiply the data polynomial by x^r and reserve r positions for the check bits.

Why is XOR used? Addition and subtraction over GF(2) both correspond to XOR.

What is the syndrome? The remainder obtained by dividing the received frame by the generator.

Does a zero syndrome prove that data is correct? No. Some changed frames are still divisible by the generator.

Does CRC correct errors? This CRC check detects errors; correction or retransmission requires another mechanism.

How many CRC bits are added? r bits, where r is the degree of the generator.

Is the example a standard CRC 16 implementation? No. It is a generic educational augmented CRC using a user entered polynomial.

References

[1] Boost C++ Libraries. CRC Library Introduction. Modulo 2 division, augmentation, check consistency and CRC parameters. https://www.boost.org/latest/doc/html/crc/introduction.html

[2] Oracle. Java Platform Standard Edition 17 javac command documentation. https://docs.oracle.com/en/java/javase/17/docs/specs/man/javac.html

[3] Oracle. Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/index.html

The project source is a teaching implementation created for this deliverable. References support the theory and Java platform usage; no external Java code library is required.